

Bases del lenguaje Javascript

Módulo: Fundamentos de desarrollo Front-End

| AE5: Codificar en lenguaje javascript rutinas simples para la personalización de eventos sencillos dentro de un documento html dando solución al problema planteado.

Introducción

En esta lección introductoria, exploraremos los fundamentos del lenguaje de programación JavaScript. Aprenderemos sobre su historia, relevancia en el desarrollo web y los conceptos básicos que nos permitirán comenzar a escribir código en JavaScript.

Aprendizaje esperado

Con estos conocimientos, estarás preparado para avanzar en tu aprendizaje de JavaScript y comenzar a construir aplicaciones web interactivas y dinámicas. Cuando finalices la lección podrás:

- Comprender la historia de JavaScript y su importancia en el desarrollo web.
- Declarar y utilizar variables en JavaScript para almacenar y manipular datos.
- Utilizar expresiones aritméticas y sentencias condicionales para realizar cálculos y tomar decisiones en el código JavaScript.
- Definir y utilizar funciones en JavaScript para agrupar y reutilizar bloques de código.
- Ejecutar código JavaScript en la consola del navegador y depurarlo para identificar y corregir errores.
- Seleccionar y manipular elementos del DOM utilizando selectores básicos en JavaScript.
- Obtener y modificar valores y textos de los elementos del DOM utilizando JavaScript.
- Trabajar con eventos básicos como `onClick` y `onChange` para responder a la interacción del usuario en una página web.

Bases del Lenguaje JavaScript

Breve historia de JavaScript

JavaScript es un lenguaje de programación que fue creado en 1995 por Brendan Eich, quien **trabajaba en Netscape Communications Corporation** en ese momento. Inicialmente, el lenguaje se llamaba "**LiveScript**" y fue desarrollado como una forma de agregar interactividad y funcionalidad a las páginas web.

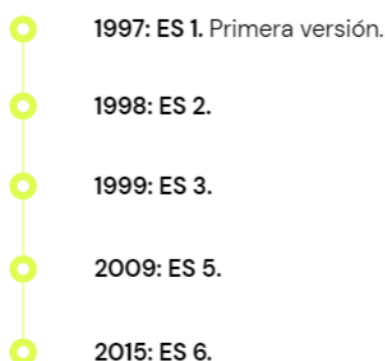
En ese momento, las páginas web se componían principalmente de contenido estático y no había una forma eficiente de agregar elementos dinámicos o interactuar con el usuario. **Con la creación de JavaScript, se abrió un nuevo mundo de posibilidades para la web.**

En 1996, Netscape presentó JavaScript junto con el lanzamiento de **Netscape Navigator 2.0**. Esto marcó el comienzo de la popularización de JavaScript, ya que se convirtió en un lenguaje de scripting incorporado en los navegadores web, lo que permitió a los desarrolladores crear páginas web más interactivas y dinámicas.

En los años siguientes, JavaScript continuó evolucionando y ganando popularidad. En 1997, **la especificación de JavaScript fue adoptada por la Organización Internacional de Normalización (ISO)** y se convirtió en el estándar ECMA-262.

Con el tiempo, **JavaScript se convirtió en un lenguaje de programación versátil que se utiliza tanto en el lado del cliente como en el lado del servidor**. Junto con HTML y CSS, JavaScript se convirtió en uno de los pilares fundamentales del desarrollo web moderno.

Evolución de JavaScript:



Hoy en día, JavaScript se utiliza ampliamente en una variedad de contextos, **incluyendo el desarrollo web, aplicaciones móviles, aplicaciones de escritorio y desarrollo de juegos**. Además, el auge de las bibliotecas y frameworks como React, Angular y Vue.js ha impulsado aún más el uso y la popularidad de JavaScript.

La evolución de JavaScript ha sido impresionante, y sigue siendo un lenguaje en constante desarrollo. Nuevas características, mejoras de rendimiento y estándares se agregan regularmente al lenguaje para mantenerlo actualizado y adaptado a las necesidades cambiantes de los desarrolladores y las tecnologías web.

JavaScript se utiliza tanto para construir aplicaciones de Frontend como de Backend. Por Frontend entendemos a la parte de la aplicación que corre en el navegador y con la cual interactúan los usuarios.

Nuestra aplicación de Frontend también consume datos y servicios ofrecidos por algún Backend. JavaScript será la herramienta que nos permitirá comunicarnos e intercambiar información con APIs u otras aplicaciones.

Algoritmos

En programación, **un algoritmo es un conjunto de procedimientos o funciones ordenados que se necesitan para realizar cierta operación o acción**. Por ejemplo, en una suma el algoritmo implica tomar un dato, sumarlo a otro y obtener un resultado.

Pensar en algoritmos es una práctica que debemos fortalecer como desarrolladores. Consiste en **encarar un problema complejo y dividir su resolución en diversos pasos, pensar cómo resolver cada uno y luego secuenciarlos correctamente para llegar al resultado esperado**.

Relevancia de Javascript

Sintaxis y código

JavaScript tiene sus propias reglas para la sintaxis, aunque respeta los estándares de muchos lenguajes de programación lógicos.

Nuestro código JavaScript se asocia al archivo HTML que se carga en el navegador. Tenemos dos maneras de escribir código JavaScript en nuestras aplicaciones web.

JavaScript es un lenguaje de programación de alto nivel que desempeña un papel fundamental en el desarrollo web y en la creación de aplicaciones interactivas en el navegador. A continuación, se destacan algunas razones que demuestran la relevancia y la importancia de JavaScript en el panorama actual:

- **Interactividad en el navegador:** JavaScript permite agregar interactividad y dinamismo a las páginas web. Gracias a este lenguaje, es posible crear efectos visuales, animaciones, juegos, validaciones de formularios y muchas otras características que mejoran la experiencia del usuario en el navegador.
- **Desarrollo web front-end:** JavaScript es el principal lenguaje utilizado en el desarrollo de la capa de presentación o front-end de las aplicaciones web. Junto con HTML y CSS, JavaScript permite construir interfaces de usuario interactivas y receptivas, proporcionando una experiencia de usuario fluida e intuitiva.
- **Frameworks y bibliotecas populares:** Existen numerosos frameworks y bibliotecas de JavaScript, como React, Angular y Vue.js, que facilitan y agilizan el desarrollo de aplicaciones web complejas. Estas herramientas proporcionan una arquitectura modular, componentes reutilizables y una forma eficiente de manejar la lógica del front-end.
- **Desarrollo web back-end:** JavaScript no se limita solo al front-end, ya que también puede utilizarse para el desarrollo del lado del servidor o back-end. Node.js, una plataforma basada en JavaScript, permite ejecutar código JavaScript en el servidor, lo que brinda la posibilidad de crear aplicaciones web completas utilizando un solo lenguaje en todos los niveles.
- **Amplia comunidad y recursos:** JavaScript cuenta con una comunidad de desarrolladores muy activa y una amplia gama de recursos disponibles. Existen numerosos foros, documentación, tutoriales y bibliotecas de código abierto que facilitan el aprendizaje y la resolución de problemas en JavaScript.
- **Compatibilidad con múltiples navegadores:** JavaScript es compatible con la mayoría de los navegadores web modernos, lo que garantiza que las aplicaciones desarrolladas en JavaScript sean accesibles y se ejecuten correctamente en diferentes entornos.
- **Integración con tecnologías emergentes:** JavaScript se ha adaptado y evolucionado para integrarse con tecnologías emergentes como la

inteligencia artificial, la realidad virtual, la realidad aumentada y el Internet de las cosas. Esto permite desarrollar aplicaciones avanzadas y aprovechar las últimas tendencias tecnológicas.

¿Cómo escribir código JS?

Dentro de un archivo html, podemos escribir código de JavaScript entre medio de las etiquetas `<script>`

Ejemplo:

```
<script>
  //aca va el código en JS
</script>
```

En un archivo individual con extensión .js podemos escribirlo de la siguiente manera:

Ejemplo: mi-archivo.js

Enlazar el archivo JavaScript al archivo HTML:

Para ejecutar el código JavaScript en un navegador web, debes enlazar el archivo JavaScript al archivo HTML correspondiente. Para ello, puedes usar la etiqueta `<script>` en el encabezado o al final del archivo HTML y especificar la ruta del archivo JavaScript. Por ejemplo:

```
<script src="ruta/al/archivo/script.js"></script>
```

Hay algunos aspectos importantes a considerar al escribir código JavaScript:

Al agregar comentarios en tu código JavaScript puedes hacerlo más legible y comprensible. Los comentarios son líneas de código que no se ejecutan y se utilizan para agregar notas o explicaciones. En JavaScript, puedes usar `//` para comentarios de una línea y `/* ... */` para comentarios de varias líneas.

Por ejemplo:

```
// Esto es un comentario de una línea

/*
  Esto es un comentario
```

```
de varias líneas  
*/
```

Funciones Nativas

Las funciones nativas, también conocidas como funciones incorporadas o funciones predefinidas, son funciones que vienen incluidas en JavaScript y están disponibles para su uso directo sin necesidad de definirlas previamente. Estas funciones proporcionan una amplia gama de funcionalidades y permiten realizar diversas tareas comunes de manera eficiente. A continuación, se mencionan algunas de las funciones nativas más utilizadas en JavaScript:

console.log(): Esta función se utiliza para imprimir mensajes en la consola del navegador o en la consola del entorno de desarrollo. Es útil para realizar pruebas y depurar el código, ya que puedes imprimir valores y mensajes para verificar el flujo de ejecución.

```
console.log("Mensaje de prueba");
```

alert(): La función alert() muestra una ventana emergente en el navegador con un mensaje para el usuario. Es útil para mostrar mensajes de advertencia, notificaciones o solicitar confirmación del usuario.

```
alert("¡Hola, mundo!");
```

confirm(): La función confirm() muestra una ventana emergente en el navegador con un mensaje y botones de confirmación ("Aceptar" y "Cancelar"). Es útil para obtener la confirmación del usuario en situaciones específicas.

```
let respuesta = confirm("¿Está seguro de eliminar este elemento?");
```

Prompt: El prompt en JavaScript es una función que muestra un cuadro de diálogo al usuario para solicitar una entrada de datos. Proporciona una forma sencilla de obtener información del usuario a través de una ventana emergente en el navegador.

```
let nombre = prompt("Por favor, ingrese su nombre:");  
console.log("Hola, " + nombre + "! Bienvenido.");
```

parseInt() y **parseFloat()**: Estas funciones se utilizan para convertir una cadena de texto en un número entero o de punto flotante, respectivamente. Son útiles cuando se necesita manipular datos numéricos ingresados como cadenas de texto.

```
let numero1 = parseInt("10");  
let numero2 = parseFloat("3.14");
```

Si el texto contiene un número con decimales el resultado será la parte entera de ese número, eliminando los decimales. Si hay espacios en blanco antes o después del número, van a ser ignorados. Si el string comienza con un número y luego le sigue otro texto, esto último es ignorado.

Si el primer carácter de ese String, no puede convertirse a número, **parseInt()** devuelve NaN, que es la sigla para la expresión "Not a Number" (no es un número). Si se pasa NaN en operaciones aritméticas, la operación resultante también será NaN.

Concatenación

La concatenación en JavaScript se refiere a la unión de dos o más cadenas de texto en una sola. Es una operación común cuando necesitas combinar texto para formar mensajes, etiquetas, sentencias, entre otros.

En JavaScript, puedes realizar la concatenación de varias formas:

Utilizando el operador de concatenación +: El operador + se utiliza para unir dos cadenas de texto. Por ejemplo:

```
let nombre = "Juan";  
let apellido = "Pérez";  
let nombreCompleto = nombre + " " + apellido;  
console.log(nombreCompleto); // Resultado: "Juan Pérez"
```

Utilizando plantillas de cadena (template literals): Las plantillas de cadena son una forma más moderna y conveniente de realizar concatenación en JavaScript. Se utilizan utilizando comillas graves (```) y permiten incrustar variables o expresiones dentro del texto utilizando \${}. Por ejemplo:


```
let nombre = "Juan";  
let mensaje = `Hola, ${nombre}!`;   
console.log(mensaje); // Resultado: "Hola, Juan!"
```

La concatenación de cadenas es una operación fundamental en JavaScript y se utiliza ampliamente en el desarrollo de aplicaciones web para generar y manipular texto dinámicamente.

Variables

Sintaxis

Asegúrate de seguir la sintaxis correcta del lenguaje JavaScript. Presta atención a los paréntesis, llaves, puntos y comas. Un error de sintaxis puede provocar que el código no funcione correctamente.

En JavaScript, **las variables se utilizan para almacenar y manipular datos**. Para declarar una variable en JavaScript, se utilizan las **palabras clave var, let o const**, seguidas del nombre de la variable. Aquí tienes un resumen de cada una de estas palabras clave:

var: Era la forma tradicional de declarar variables en JavaScript, pero se ha vuelto menos común con la introducción de let y const. **Las variables declaradas con var tienen un alcance de función o global**, lo que significa que están disponibles en todo el ámbito de la función o en todo el programa si se declaran fuera de una función.

```
var edad = 25;
```

let: Introducido en ECMAScript 6 (ES6), **let permite declarar variables con un alcance de bloque**. Las variables declaradas con let están limitadas al bloque en el que se declaran, como un bloque de código dentro de una función, un bucle for o un bloque if.

```
let nombre = 'Juan';  
const nacionalidad = 'Argentina';
```

const: También introducido en ES6, `const` se utiliza para declarar constantes, es decir, variables que no pueden cambiar su valor una vez asignado. Al igual que `let`, `const` tiene un alcance de bloque.

```
const pi = 3.1416;
```

Es importante tener en cuenta que **let** y **const** siguen las mejores prácticas actuales para declarar variables en JavaScript. Se recomienda utilizar **const siempre que sea posible**, ya que ayuda a prevenir la reasignación accidental de valores. **Si sabes que el valor de una variable cambiará, puedes utilizar let.**

Además de declarar variables, puedes asignarles valores utilizando el operador de asignación `=`:

```
let mensaje = 'Hola, mundo!';
```

Una vez que una variable ha sido declarada, puedes utilizarla en tu código, ya sea para realizar operaciones, imprimir su valor en la consola o utilizarla en otras expresiones.

```
let x = 10;
let y = 5;
let suma = x + y;
console.log(suma); // Imprime 15 en la consola
```

Declaración

La declaración de variables en JavaScript implica crear una variable utilizando las palabras clave **var**, **let** o **const**, seguidas del nombre deseado para la variable. Es importante seguir algunas reglas al nombrar las variables:

No se deben utilizar espacios en el nombre de la variable. En su lugar, se pueden utilizar guiones bajos (`_`) o notación camelCase (la primera letra de cada palabra, excepto la primera, se escribe en mayúscula).

```
var miVariable;
let nombreCompleto;
```

```
const numeroDeTelefono;
```

Los nombres de las variables no pueden comenzar con un número. Deben comenzar con una letra, un guión bajo (_) o un signo de dólar (\$).

```
var _precio;  
let $cantidad;
```

Evita el uso de caracteres especiales, como !, @, #, \$, %, etc. en los nombres de las variables. Solo se permiten letras, números, guiones bajos (_) y signos de dólar (\$).

```
var cantidad_total;  
let usuario2;  
const MAXIMO_VALOR;
```

Siguiendo estas reglas, puedes declarar tus variables de manera adecuada y asegurarte de que sean reconocidas y utilizables en tu código JavaScript. Recuerda también elegir la palabra clave adecuada (var, let o const) según tus necesidades de alcance y mutabilidad de la variable.

Asignación

La asignación de valores a una variable, se realiza utilizando el operador de asignación, que **es el signo igual (=)**. Este operador asigna el valor a la variable en el lado izquierdo de la expresión.

```
var nombre = "Juan"; // Asigna el valor "Juan" a la variable nombre  
let edad = 25; // Asigna el valor 25 a la variable edad  
const pi = 3.1416; // Asigna el valor 3.1416 a la constante pi
```

Es importante tener en cuenta que en JavaScript, el tipo de dato de una variable no está predefinido y puede cambiar durante la ejecución del programa. Esto se conoce como tipado dinámico. Por ejemplo, puedes asignar inicialmente un valor numérico a una variable y luego cambiar su valor a una cadena de texto más adelante en el código.

```
let numero = 10; // Asigna el valor 10 a la variable numero  
  
numero = "veinte"; // Cambia el valor de la variable numero a la cadena de  
texto "veinte"
```

La asignación de valores a una variable se realiza de derecha a izquierda, donde el valor en el lado derecho se asigna a la variable en el lado izquierdo.

Es importante tener en cuenta que el operador de asignación (=) no debe confundirse con el operador de igualdad (== o ===) que se utiliza para comparar valores.

Inicializar variables

En JavaScript es posible declarar una variable y asignarle un valor inicial en el mismo proceso. Esto se conoce como inicialización de variable.

```
var edad = 25; // Declaración y asignación inicial de la variable edad  
  
let nombre = "Juan"; // Declaración y asignación inicial de la variable nombre  
  
const pi = 3.1416; // Declaración y asignación inicial de la constante pi
```

En el ejemplo anterior, se declaran las variables edad y nombre, y se les asigna un valor inicial en el mismo momento de la declaración. De manera similar, se declara y se asigna un valor inicial a la constante pi.

La inicialización de variables en el mismo proceso puede ser útil cuando conoces el valor inicial que deseas asignar a la variable al momento de declararla. Esto evita la necesidad de realizar una asignación por separado después de la declaración.

Es importante tener en cuenta que al inicializar una variable, el valor inicial puede ser de cualquier tipo de dato válido en JavaScript, como números, cadenas de texto, booleanos, arreglos, objetos, etc.

```
let temperatura = 25.5; // Declaración y asignación inicial de la variable temperatura  
(tipo número)  
  
const nombreCompleto = "María Pérez"; // Declaración y asignación inicial de la  
constante nombreCompleto (tipo cadena de texto)
```

```
var esActivo = true; // Declaración y asignación inicial de la variable esActivo (tipo booleano)
```

Al declarar e inicializar variables, es recomendable seguir las mejores prácticas de nomenclatura y elegir nombres descriptivos que reflejen el propósito o la naturaleza de la variable.

Expresiones aritméticas

En JavaScript, las expresiones aritméticas se utilizan para realizar operaciones matemáticas y calcular resultados numéricos. Las expresiones aritméticas pueden involucrar operadores matemáticos como suma (+), resta (-), multiplicación (*), división (/), resto (%) y exponente (**).

```
let x = 5;
let y = 3;

let suma = x + y; // Suma: 8
let resta = x - y; // Resta: 2
let multiplicacion = x * y; // Multiplicación: 15
let division = x / y; // División: 1.6666666666666667
let resto = x % y; // Resto: 2
let exponente = x ** y; // Exponente: 125
```

En el ejemplo anterior, se utilizan diferentes operadores aritméticos para realizar operaciones matemáticas básicas. Es importante tener en cuenta que las operaciones se realizan de acuerdo con las reglas de precedencia matemática estándar.

Además de los operadores aritméticos básicos, JavaScript también proporciona operadores de asignación compuestos que permiten combinar una operación aritmética con una asignación en una sola expresión.

Vamos a usar las 4 operaciones matemáticas básicas, suma, resta, multiplicación y división desde la consola. La sintaxis de Javascript para el uso de estas operaciones es igual a la que usamos en cualquier calculadora.

A medida que vayas desarrollando habilidades de programación te vas a dar cuenta de que podemos aplicarlas en muchas cosas más.

Suma (+)

La operación suma se realiza concatenando los números que queremos sumar con el signo de la suma (+).

```
let numero1 = 10;
let numero2 = 50;
console.log(10 + 50);
console.log(numero1 + numero2)

let resultadoSuma = numero1 + numero2;
console.log(resultadoSuma)
```

Resta (-)

De manera análoga, la operación resta se realiza concatenando los números que queremos restar con el signo de la resta (-).

```
let numero1 = 100;
let numero2 = 50;
console.log(100 - 50);
console.log(numero1 - numero2)

let resultadoResta = numero1 - numero2;
console.log(resultadoResta)
```

Multiplicación (*)

La operación multiplicación se realiza concatenando los números que queremos multiplicar con el signo de la multiplicación, el asterisco (*).

```
let numero1 = 70;
let numero2 = 70;
console.log(70 * 7);
console.log(numero1 * numero2)

let resultadoMultiplicacion = numero1 * numero2;
console.log(resultadoMultiplicacion)
```

División (/)

La operación división se realiza ubicando signo de la división, la barra inclinada hacia la derecha (/), entre los números que queremos dividir.

```
let numero1 = 80;
let numero2 = 4;
console.log(80 / 4);
console.log(numero1 / numero2)

let resultadoDivision = numero1 / numero2;
console.log(resultadoDivision)
```

% (módulo): Calcula el resto de la división entre x e y. En el ejemplo, x % y da como resultado 2, que es el resto de la división de 5 entre 3.

**** (exponente):** Eleva x a la potencia y. En el ejemplo, x ** y da como resultado 125, que es 5 elevado a la potencia 3.

```
let resto = x % y; // Resto: 2
```

```
let exponente = x ** y; // Exponente: 125
```

Otras operaciones

Además de las operaciones aritméticas básicas, existe otro tipo de operaciones encargadas de convertir tipos de datos string a un valor numérico.

Operadores de incremento y decremento

Existen otros dos operadores aritméticos que aportan la utilidad de sumar o restar 1 unidad a una variable.

Cuando queremos aumentar o disminuir el valor numérico de una variable es común realizar lo siguiente:

```
let numero = 100;

console.log(numero)//Salida por consola --> 100

//incremento

numero = numero + 1;

console.log(numero)//Salida por consola --> 101

//decremento

numero = numero - 1;

console.log(numero)//Salida por consola --> 100
```

Existe un método abreviado de hacer esto. El signo que realiza la adición en **la suma se escribe ++** (con dos signos más seguidos) y en el caso la resta se escribe -- (dos signos menos seguidos).

```
let numero = 100;

console.log(numero)//Salida por consola --> 100

//incremento

numero++;

console.log(numero)//Salida por consola --> 101
```



```
//decremento

numero = numero--;

console.log(numero)//Salida por consola --> 100
```

Estos dos operadores nos van a ser de gran utilidad cuando empecemos a ver Bucles.

Operadores de Asignación

Además de los operadores aritméticos básicos, JavaScript también proporciona operadores de asignación compuestos que permiten combinar una operación aritmética con una asignación en una sola expresión. Por ejemplo:

```
let z = 10;

z += 5; // Equivale a z = z + 5; (z = 15)

z -= 3; // Equivale a z = z - 3; (z = 12)

z *= 2; // Equivale a z = z * 2; (z = 24)

z /= 4; // Equivale a z = z / 4; (z = 6)

z %= 2; // Equivale a z = z % 2; (z = 0)
```

En este caso, se utilizan operadores de asignación compuestos junto con los operadores aritméticos básicos para actualizar el valor de la variable z realizando la operación aritmética correspondiente y asignando el resultado a la misma variable.

Operadores de comparación

Los operadores de comparación en JavaScript **se utilizan para comparar dos valores y devolver un resultado booleano (true o false) que indica si la comparación es verdadera o falsa.**

Igualdad (==): Compara si dos valores son iguales, sin tener en cuenta el tipo de dato. Devuelve true si son iguales y false si no lo son.

```
let x = 5;

let y = "5";
```

```
console.log(x == y); // Devuelve true
```

En el ejemplo, la comparación `x == y` devuelve `true` porque ambos valores son considerados iguales, a pesar de que uno es un número y el otro es una cadena de texto.

Igualdad estricta (===): Compara si dos valores son iguales, teniendo en cuenta tanto el valor como el tipo de dato. Devuelve `true` si son iguales en valor y tipo, y `false` si no lo son.

```
let x = 5;  
let y = "5";  
  
console.log(x === y); // Devuelve false
```

En este caso, la comparación `x === y` devuelve `false` porque los valores son iguales, pero los tipos de dato son diferentes.

Desigualdad (!= o !==): Compara si dos valores no son iguales. Devuelve `true` si son diferentes y `false` si son iguales. **El operador `!==` también verifica el tipo de dato.**

```
let x = 5;  
let y = 10;  
  
console.log(x != y); // Devuelve true  
console.log(x !== y); // Devuelve true
```

En el ejemplo, tanto la comparación `x != y` como `x !== y` devuelven `true` porque los valores son diferentes.

Otros operadores de comparación: Además de la igualdad y desigualdad, existen otros operadores de comparación como `>`, `<`, `>=` y `<=`, que comparan si un valor es mayor, menor, mayor o igual, o menor o igual, respectivamente.

```
let x = 5;  
  
let y = 10;  
  
console.log(x > y); // Devuelve false  
console.log(x < y); // Devuelve true  
console.log(x >= y); // Devuelve false  
console.log(x <= y); // Devuelve true
```

En el ejemplo, la comparación $x < y$ devuelve true porque el valor de x es menor que el valor de y.

Operadores Lógicos

Los operadores lógicos en JavaScript se utilizan para combinar o invertir valores booleanos y realizar operaciones lógicas.

Operador AND (&&): El operador AND devuelve true si ambos operandos son true, y devuelve false en cualquier otro caso.

```
let x = 5;  
  
let y = 10;  
  
console.log(x > 0 && y > 0); // Devuelve true  
console.log(x > 0 && y < 0); // Devuelve false
```

En el primer ejemplo, $x > 0$ y $y > 0$ son ambas expresiones verdaderas, por lo que la expresión completa $x > 0 \ \&\& \ y > 0$ devuelve true. En el segundo ejemplo, $y < 0$ es una expresión falsa, por lo que la expresión completa $x > 0 \ \&\& \ y < 0$ devuelve false.

Operador OR (||): El operador OR devuelve true si al menos uno de los operandos es true, y devuelve false solo si ambos operandos son false.

```
let x = 5;
```

```
let y = 10;

console.log(x > 0 || y > 0); // Devuelve true
console.log(x < 0 || y < 0); // Devuelve false
```

En el primer ejemplo, $x > 0$ es una expresión verdadera, por lo que la expresión completa $x > 0 \parallel y > 0$ devuelve true. En el segundo ejemplo, $x < 0$ y $y < 0$ son ambas expresiones falsas, por lo que la expresión completa $x < 0 \parallel y < 0$ devuelve false.

Operador NOT (!): El operador NOT invierte el valor de una expresión booleana. Si la expresión es true, el operador NOT devuelve false, y si la expresión es false, el operador NOT devuelve true.

```
let x = 5;
let y = 10;

console.log(!(x > 0)); // Devuelve false
console.log(!(y < 0)); // Devuelve true
```

En el primer ejemplo, $x > 0$ es una expresión verdadera, pero el operador NOT invierte su valor a false. En el segundo ejemplo, $y < 0$ es una expresión falsa, y el operador NOT la invierte a true.

Estos operadores lógicos te permiten combinar expresiones y realizar operaciones lógicas en JavaScript. puedes utilizarlos para tomar decisiones basadas en múltiples condiciones o para invertir el valor de una expresión booleana.

Sentencias condicionales

Las sentencias condicionales en JavaScript nos permiten tomar decisiones basadas en una condición específica. Dependiendo del resultado de esa condición, se ejecutará un bloque de código u otro.

IF - ELSE: Es un tipo de estructura que existe en muchos lenguajes de programación y permite que el programa resuelve por su cuenta preguntas o decisiones para continuar con el flujo de tareas del usuario. Es una condicional.

Por ejemplo:

Si el usuario está logueado al sitio, accede a cierto contenido. Si el usuario no está logueado, no accede al contenido. En este caso la condición es si está logueado o no y en base a esa respuesta (si o no, if o else) se mostrará un contenido u otro.

```
// Variable que indica si el usuario está logueado
let usuarioLogueado = true;

if (usuarioLogueado) {
    // Si el usuario está logueado, se muestra el contenido para usuarios
    logueados
    console.log("Bienvenido, usuario logueado. Acá está el contenido exclusivo.");
} else {
    // Si el usuario no está logueado, se muestra el contenido para usuarios no
    logueados
    console.log("Por favor, inicia sesión para acceder a este contenido.");
}
```

La sentencia if se utiliza para ejecutar un bloque de código si la condición especificada es evaluada como verdadera (true).

```
let edad = 18;

if (edad >= 18) {

  console.log("Eres mayor de edad");

}
```

En el ejemplo, si la variable edad es mayor o igual a 18, se ejecutará el bloque de código dentro de las llaves {} y se imprimirá el mensaje "Eres mayor de edad".

La sentencia if-else nos permite ejecutar un bloque de código si la condición especificada es verdadera (true), y otro bloque de código si la condición es falsa (false).

```
let hora = 13;

if (hora < 12) {

  console.log("Buenos días");

} else {

  console.log("Buenas tardes");

}
```

En el ejemplo, si la variable hora es menor a 12, se imprimirá "Buenos días"; de lo contrario, se imprimirá "Buenas tardes".

La sentencia if-else if-else nos permite evaluar múltiples condiciones y ejecutar diferentes bloques de código en función de esas condiciones.

```
let puntaje = 80;

if (puntaje >= 90) {
```

```
console.log("Excelente");  
} else if (puntaje >= 70) {  
    console.log("Aprobado");  
} else {  
    console.log("Reprobado");  
}
```

En el ejemplo, se evalúa el puntaje y se imprime un mensaje según el resultado. En JavaScript, puedes utilizar los operadores lógicos `&&` (AND) y `||` (OR) en las sentencias condicionales para evaluar múltiples condiciones.

Operador AND (&&) con IF/ELSE:

El operador `&&` devuelve `true` si ambas condiciones son verdaderas. Si alguna de las condiciones es falsa, devuelve `false`.

```
let edad = 25;  
let tieneLicencia = true;  
  
if (edad >= 18 && tieneLicencia) {  
    console.log("puedes conducir legalmente.");  
} else {  
    console.log("No cumples con los requisitos para conducir.");  
}
```

En el ejemplo, la condición dentro del `if` utiliza el **operador `&&`** para verificar si la edad es mayor o igual a 18 y **si `tieneLicencia` es `true`**. Si ambas condiciones son verdaderas, se muestra el mensaje "puedes conducir legalmente". Si alguna de las condiciones es falsa, se muestra el mensaje "No cumples con los requisitos para conducir".

Operador OR (||) con IF/ELSE:

El operador || devuelve true si al menos una de las condiciones es verdadera. Solo devuelve false si ambas condiciones son falsas.

```
let esEstudiante = false;
let esEmpleado = true;

if (esEstudiante || esEmpleado) {
  console.log("Tenes acceso a descuentos especiales.");
} else {
  console.log("No tenes acceso a descuentos especiales.");
}
```

En este ejemplo, la condición dentro del if utiliza el **operador || para verificar si esEstudiante es true o si esEmpleado es true**. Si al menos una de las condiciones es verdadera, se muestra el mensaje "Tenes acceso a descuentos especiales". Si ambas condiciones son falsas, se muestra el mensaje "No tenes acceso a descuentos especiales".

Estos operadores lógicos (&& y ||) son muy útiles para combinar múltiples condiciones en una sentencia condicional y tomar decisiones basadas en diferentes escenarios.

Operador Ternario

Ahora que sabemos como funciona la lógica de los condicionales, podríamos “achicar” un poco más esas funciones. Es decir, podemos escribir la misma instrucción pero de un modo más corto y prolijo. El operador ternario, también conocido como operador condicional, es un operador que permite realizar una evaluación condicional de forma más concisa y expresiva en JavaScript.

El operador ternario consta de tres partes:

1. La condición es una expresión que se evalúa como verdadera (true) o falsa (false).

2. La expresión1 se ejecuta si la condición es verdadera.
3. La expresión2 se ejecuta si la condición es falsa.

Su sintaxis es la siguiente:

```
condicion ? expresion1 : expresion2;
```

Sigamos con este ejemplo:

```
let num1 = 7;
let num2 = 5;

if(num1 > num2 ){
  console.log("El número 1 es mayor") }
else {
  console.log("El número 2 es mayor")}
```

Vamos a tener que reemplazar algunas palabras claves por símbolos para incorporar el operador ternario.

```
let num1 = 7;
let num2 = 5;

//Operador ternario: (condición ? verdadero : falso)

let rta = num1 > num2 ? "el 1" : "el 2";

console.log(rta);
```

Entonces, ¿Qué cambiamos?

Para utilizar los operadores ternarios, ya no vamos a escribir if...else, pero, su estructura se mantiene. Es decir, tenemos una condición (num1 > num2), nuestro resultado por si esta condición es verdadero, lo asignamos utilizando el signo (?). Y para terminar, los dos puntos (:) nos permiten identificar cuál será su resultado falso.

Switch

La **sentencia switch** en JavaScript se utiliza cuando se necesita **evaluar una expresión y ejecutar diferentes bloques de código** según el valor de esa expresión. Es una forma más concisa y estructurada de manejar múltiples casos que cumplen ciertas condiciones.

Sintaxis básica:

```
switch (expresion) {  
  
  case valor1:  
  
    // Bloque de código a ejecutar si la expresion es igual a valor1  
  
    break;  
  
  case valor2:  
  
    // Bloque de código a ejecutar si la expresion es igual a valor2  
  
    break;  
  
  case valor3:  
  
    // Bloque de código a ejecutar si la expresion es igual a valor3  
  
    break;  
  
  // ...  
  
  default:  
  
    // Bloque de código a ejecutar si ninguno de los casos anteriores se cumple  
  
}
```

La expresión es evaluada una vez y luego se compara con cada uno de los casos especificados. **Si hay una coincidencia entre la expresión y un caso**, se ejecuta el bloque de código correspondiente. Es importante destacar que es necesario

utilizar la palabra clave `break` después de cada bloque de código para salir del `switch` y evitar que se ejecuten los casos siguientes.

Si ninguno de los casos coincide con el valor de la expresión, **se ejecuta el bloque de código dentro del default**. El bloque `default` es opcional y se utiliza para manejar el caso en que no se cumpla ninguna de las condiciones anteriores.

```
let dia = "lunes";

switch (dia) {
  case "lunes":
    console.log("Hoy es lunes");
    break;
  case "martes":
    console.log("Hoy es martes");
    break;
  case "miércoles":
    console.log("Hoy es miércoles");
    break;
  case "jueves":
    console.log("Hoy es jueves");
    break;
  case "viernes":
    console.log("Hoy es viernes");
    break;
  default:
    console.log("Hoy no es ninguno de los días de la semana");
}
```

En este ejemplo, se evalúa el valor de la variable `dia` y se ejecuta el bloque de código correspondiente al caso que coincida. Si el valor de `dia` es "lunes", se muestra "Hoy es lunes" en la consola. Si no se cumple ninguno de los casos anteriores, se ejecuta el bloque dentro del default y se muestra "Hoy no es ninguno de los días de la semana".

La sentencia `switch` es útil cuando se tienen múltiples casos que deben ser evaluados de manera independiente y se desea ejecutar un bloque de código específico según el valor de la expresión evaluada.

Funciones 🙌

Cuando se desarrolla una aplicación o sitio web, es muy habitual utilizar una y otra vez las mismas instrucciones.

En programación, una función es un conjunto de instrucciones que se agrupan para realizar una tarea concreta, que luego se puede reutilizar a lo largo de diferentes instancias del código.

Las funciones en JavaScript son bloques de código reutilizables que se pueden invocar para realizar una tarea específica. Proporcionan una forma de encapsular un conjunto de instrucciones y ejecutarlas en cualquier momento que se necesiten. Las funciones pueden aceptar argumentos (parámetros) y devolver un resultado.

Las principales ventajas del uso de funciones son:

- Evita instrucciones duplicadas (Principio DRY).
- Soluciona un problema complejo usando tareas sencillas (Principio KISS).
- Focaliza tareas prioritarias para el programa (Principio YAGNI).
- Aporta ordenamiento y entendimiento al código.
- Aporta facilidad y rapidez para hacer modificaciones.

Declaración

La declaración de una función en JavaScript se realiza utilizando la palabra **clave function**, seguida del **nombre de la función** y un **par de paréntesis ()**. Dentro de los paréntesis, puedes especificar los parámetros que la función puede aceptar, separados por comas. Luego, se abre un bloque de código con llaves `{}` donde se define el cuerpo de la función, es decir, las instrucciones que se ejecutarán cuando la función sea invocada.

```
function saludar() {  
  //Bloque de código  
  console.log("¡Hola estudiantes!");  
}
```

Una vez que declaramos la función, podemos usarla en cualquier otra parte del código todas las veces que queramos.

Para ejecutar una función sólo hay que escribir su nombre y finalizar la sentencia con (). **A esto se lo conoce como llamado a la función.** Donde escribamos el llamado, se interpretarán las instrucciones definidas en esa función.

```
saludar();
```

Parámetros

Una función simple, puede no necesitar ningún dato para funcionar. Pero cuando empezamos a codificar funciones más complejas, nos encontramos con la necesidad de recibir cierta información. Cuando enviamos a la función uno o más valores para ser empleados en sus operaciones, estamos hablando de los parámetros de la función.

Los parámetros se envían a la función mediante variables y se colocan entre los paréntesis posteriores al nombre de la función.

```
//ejemplo de sintaxis  
function conParametros(parametro1, parametro2) {  
  console.log(parametro1 + " " + parametro2);  
}  
  
//sintaxis de una función con parametros  
function saludar(nombre, edad) {
```

```
console.log("¡Hola, " + nombre + "! Tienes " + edad + " años.");
}
```

Así, podemos armar funciones dinámicas que, siguiendo la lógica que queramos, pueden generar distintos resultados al recibir diferentes valores.

Cuando invocas una función que tiene parámetros, debes proporcionar los valores correspondientes a esos parámetros. El valor que toman estos parámetros se definen en el llamado. Cuando llamamos a la función, los valores que pasamos a la función entre paréntesis se asignan posicionalmente a los parámetros correspondientes, generando posibles resultados diferentes:

```
saludar("Juan", 30); // Muestra "¡Hola, Juan! Tienes 30 años."
```

En este caso, el primer string que pasamos se asigna en parametro1, y el segundo string en parametro2; armando las salidas según la lógica definida.

Ejemplo función Sumar y Mostrar

```
//Declaración de variable para guardar el resultado de la suma
let resultado = 0;

//Función que suma dos números y asigna a resultado
function sumar(primerNumero, segundoNumero) {
    resultado = primerNumero + segundoNumero
}

//Función que muestra resultado por consola
function mostrar(mensaje) {
    console.log(mensaje)
}

//Llamamos primero a sumar y luego a mostrar
sumar(6, 3);

mostrar(resultado);
```

Resultado de una función

En el ejemplo anterior sumamos dos números a una variable declarada anteriormente. Pero las funciones pueden generar un valor de retorno usando la palabra `return`, obteniendo el valor cuando la función es llamada.

```
function sumar(primerNumero, segundoNumero) {  
    return primerNumero + segundoNumero;  
}  
  
let resultado = sumar(5, 8);
```

La función puede comportarse como una operación que genera valores (como en las operaciones matemáticas y lógicas previas). En el espacio donde se llama a la función se genera un nuevo valor: este valor es el definido por el **return** de la misma.

```
let resultado = sumar(5, 8);  
  
console.log(resultado) // ⇒ 13
```

Función calculadora:

```
function calculadora() {  
    let primerNumero = parseFloat(prompt("Ingrese el primer número:"));  
    let operacion = prompt("Ingrese la operación (+, -, *, /):");  
    let segundoNumero = parseFloat(prompt("Ingrese el segundo número:"));  
  
    let resultado;  
  
    switch (operacion) {  
        case "+":  
            resultado = primerNumero + segundoNumero;  
            break;
```

```

case "-":
    resultado = primerNumero - segundoNumero;
    break;
case "*":
    resultado = primerNumero * segundoNumero;
    break;
case "/":
    resultado = primerNumero / segundoNumero;
    break;
default:
    alert("Operación inválida");
    return; // Salir de la función si la operación es inválida
}

alert("El resultado es: " + resultado);
}

calculadora();

```

La función calculadora utiliza prompt para solicitar al usuario que ingrese el primer número, la operación y el segundo número. Luego, se realiza la operación matemática correspondiente utilizando la sentencia switch y se asigna el resultado a la variable resultado. Finalmente, se utiliza alert para mostrar el resultado al usuario.

Tené en cuenta que se utilizó parseFloat para convertir las cadenas ingresadas por el usuario en números de tipo float. Además, se agregó una validación para manejar el caso en que se ingrese una operación inválida.

Al llamar a la función calculadora, se ejecutará el código y se mostrarán los mensajes de alerta con el resultado de la operación.

Scope

El scope o ámbito de una variable es la zona del programa en la cual se define, el contexto al que pertenece la misma dentro de un algoritmo, restringiendo su uso y alcance.

JavaScript define dos ámbitos para las variables:

- Global
- Local.

Ámbito global: Las variables declaradas fuera de cualquier función tienen un ámbito global. Esto significa que están disponibles en todo el programa y pueden ser accedidas desde cualquier parte. Estas variables se conocen como variables globales y se definen utilizando las palabras clave `var`, `let` o `const` fuera de cualquier función.

```
let nombre = "Juan"; // Variable global

function saludar() {
  console.log("Hola, " + nombre); // Accediendo a la variable global dentro de la función
}

saludar(); // Imprime "Hola, Juan"
console.log(nombre); // Imprime "Juan"
```

Ámbito local: Las variables declaradas dentro de una función tienen un ámbito local, lo que significa que solo están disponibles dentro de esa función. Estas variables se conocen como variables locales y se definen utilizando las palabras clave `var`, `let` o `const` dentro de una función.

```
function saludar() {
  var mensaje = "Hola, bienvenido"; // Variable local
```

```
console.log(mensaje);  
}  
  
saludar(); // Imprime "Hola, bienvenido"  
console.log(mensaje); // Error: mensaje is not defined
```

✖ ▶ Uncaught ReferenceError: mensaje is not defined

Es importante tener en cuenta el alcance de las variables para evitar colisiones de nombres y asegurar un correcto funcionamiento del programa. Es una buena práctica limitar el alcance de las variables en la medida de lo posible utilizando ámbitos locales y evitando la declaración de variables globales innecesarias.

```
let nombre = "John Doe" // variable global  
  
function saludar() {  
  let nombre = "Juan Lopez" // variable local  
  console.log(nombre)  
}  
  
//Accede a nombre global  
console.log(nombre) // → "John Doe"  
  
//Accede a nombre local  
saludar() // → "Juan Lopez"
```

Entender que cada scope local es un espacio cerrado nos permite crear bloques de trabajo bien diferenciados e independientes, sin preocuparnos por repetir nombres de variables, sabiendo que se entienden como diferentes según donde las llamemos.

```
function sumar(num1, num2) {  
    let resultado = num1 + num2  
    return resultado  
}  
  
function restar(num1, num2) {  
    let resultado = num1 - num2  
    return resultado  
}
```

Funciones anónimas

Las funciones anónimas en JavaScript son funciones que no tienen un nombre definido. Se definen directamente como expresiones de función sin asignarles un nombre específico. Estas funciones se utilizan comúnmente en situaciones donde solo se requiere una función temporal o como argumentos para otras funciones. Una función anónima es una función que se define sin nombre y se utiliza para ser pasada como parámetro o asignada a una variable. En el caso de asignarla a una variable, pueden llamar usando el identificador de la variable declarada.

```
//Generalmente, las funciones anónimas se asignan a variables declaradas  
como constantes  
  
const suma = function (a, b) { return a + b }  
const resta = function (a, b) { return a - b }  
  
console.log( suma(15,20) )  
console.log( resta(15,5) )
```

Las funciones anónimas **son útiles cuando se necesitan funciones temporales** o cuando se desea utilizar una función como argumento en otra función, como **en el caso de funciones de devolución de llamada (callback functions) o funciones de orden superior (higher-order functions)**.

Es importante tener en cuenta que, aunque las funciones anónimas no tienen un nombre específico, se asignan a variables u otros elementos para poder ser utilizadas posteriormente.

Funciones Flechas =>

Las funciones flecha (**arrow functions**) son una forma más concisa y expresiva de definir funciones en JavaScript. Se introdujeron en ECMAScript 6 y proporcionan una sintaxis simplificada para crear funciones. Identificamos a las funciones flechas como funciones anónimas de sintaxis simplificada. No usan la palabra function pero usa => (flecha) entre los parámetros y el bloque.

Sintaxis básica:

```
const miFuncion = () => {  
  // Código de la función  
};
```

Se declara una variable miFuncion y se le asigna una función flecha como valor. El cuerpo de la función se encuentra dentro de los corchetes {}, donde se pueden escribir las operaciones que realizará la función.

```
const suma = (a, b) => { return a + b }  
  
//Si es una función de una sola línea con retorno podemos evitar escribir el  
cuerpo.  
  
const resta = (a, b) => a - b;  
  
console.log( suma(15,20) )  
console.log( resta(20,5) )
```

Si la función flecha tiene un solo parámetro, se pueden omitir los paréntesis alrededor del parámetro.

```
const duplicar = numero => {  
  return numero * 2;  
};
```

```
console.log(duplicar(5)); // Imprime 10
```

En este caso, la función flecha `duplicar` acepta un parámetro `numero` y devuelve el resultado de multiplicar ese número por 2.

Callback

Los callbacks son funciones que se pasan como argumentos a otras funciones. Son una forma común de lograr la ejecución asíncrona y la programación basada en eventos en JavaScript.

La idea principal detrás de los callbacks es permitir que una función sea llamada dentro de otra función después de que se haya completado una tarea o evento específico. Esto permite una programación más flexible y dinámica, ya que puedes definir el comportamiento que se debe ejecutar después de ciertas operaciones.

```
function operacionAsincrona(callback) {  
  // Realizar una operación asíncrona  
  // Una vez que la operación se haya completado, llamar al callback  
  callback();  
}  
  
function miCallback() {  
  console.log("La operación asíncrona se ha completado.");  
}  
  
operacionAsincrona(miCallback);
```

Ejercicio: Calcular precio

Utiliza funciones flecha para realizar operaciones matemáticas y calcular un nuevo precio basado en el precio del producto, el IVA y un descuento.

```
const suma = (a,b) => a + b
```

```
const resta = (a,b) => a - b

//Si una función es una sola línea con retorno y un parámetro puede evitar
escribir los ()

const iva = x => x * 0.21

let precioProducto = 500

let descuento = 50

//Calculo el precioProducto + IVA - descuento

let nuevoPrecio = resta(suma(precioProducto, iva(precioProducto)), descuento)

console.log(nuevoPrecio)
```

En este código, se definen las funciones flecha suma, resta e iva para realizar las operaciones matemáticas correspondientes.

La función suma toma dos parámetros a y b y devuelve la suma de los mismos. La función resta toma dos parámetros a y b y devuelve la resta de a menos b. La función iva toma un parámetro x y devuelve el resultado de multiplicar x por 0.21, que representa el 21% de IVA.

Luego, se declaran las variables precioProducto y descuento con los valores respectivos de 500 y 50.

Finalmente, se calcula el nuevoPrecio utilizando las funciones flecha y las operaciones matemáticas. Primero se aplica el iva al precioProducto utilizando la función iva(precioProducto). Luego se suma el precioProducto con el resultado anterior y se resta el descuento.

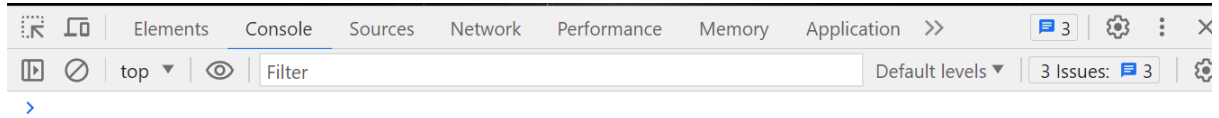
El resultado final se asigna a la variable nuevoPrecio y se muestra en la consola utilizando console.log(nuevoPrecio).

Cómo ejecutar código Javascript en la consola

Para ejecutar código JavaScript en la consola del navegador y realizar la depuración del código, puedes seguir estos pasos:

1. Abre cualquier página web en tu navegador.
2. Haz clic derecho en la página y selecciona "Inspeccionar" o presiona la tecla F12 para abrir las herramientas de desarrollo del navegador.

3. En la pestaña "Consola" de las herramientas de desarrollo, verás un prompt donde puedes ingresar y ejecutar código JavaScript. Acá puedes escribir cualquier instrucción o función en JavaScript y presionar "Enter" para ejecutarla.



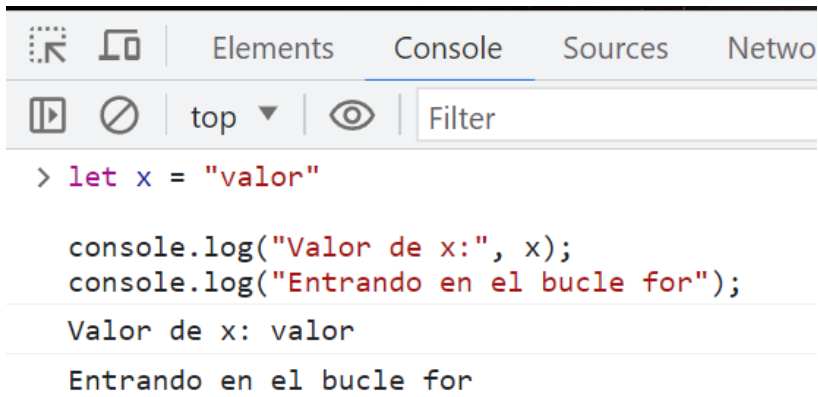
4. Observa los resultados o mensajes de error que se muestran en la consola después de ejecutar el código. puedes utilizar la consola para imprimir valores, realizar cálculos, interactuar con el DOM de la página y depurar problemas en tu código.
5. La consola del navegador es una herramienta muy útil para probar y depurar código JavaScript. Te permite ejecutar código en tiempo real y ver los resultados de tus instrucciones. Además, puedes utilizar métodos específicos de la consola, como `console.log()`, para imprimir valores y mensajes en la consola y facilitar el seguimiento de tu código durante la depuración.

Recuerda que la consola del navegador es una herramienta de desarrollo y no debe ser utilizada en producción. Es útil para propósitos de depuración y prueba durante el desarrollo de tus aplicaciones web.

Depurando el código Javascript con la consola 📌

La consola del navegador también es una herramienta muy útil para depurar el código JavaScript y encontrar errores o problemas en tu programa. puedes utilizar varias técnicas para depurar y examinar el código en la consola:

1. **Mostrar mensajes de depuración:** puedes utilizar `console.log()` para imprimir mensajes de depuración en la consola. puedes agregar mensajes en diferentes partes de tu código para verificar el valor de variables, comprobar si ciertas condiciones se cumplen o simplemente seguir el flujo de ejecución del programa. Por ejemplo:



The screenshot shows the Chrome DevTools Console with the 'Console' tab selected. The code being executed is:

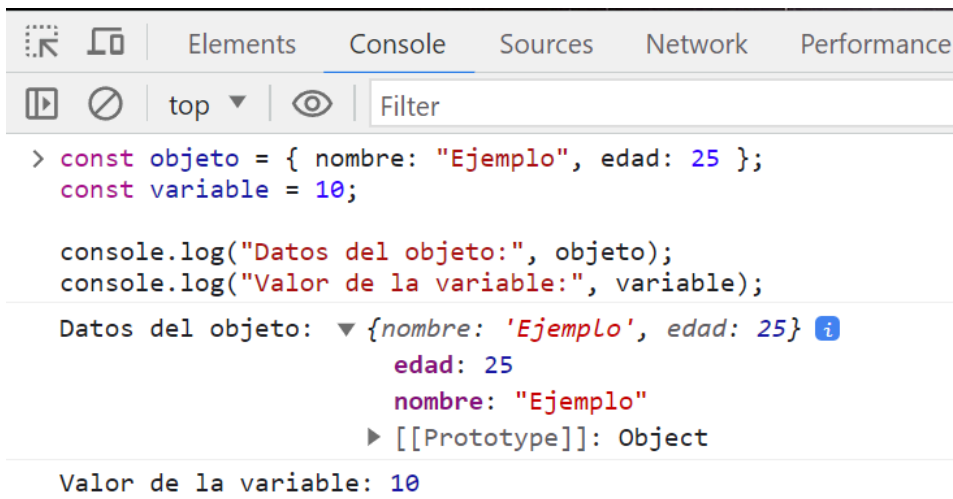
```
> let x = "valor"

console.log("Valor de x:", x);
console.log("Entrando en el bucle for");
```

The output displayed in the console is:

```
Valor de x: valor
Entrando en el bucle for
```

2. **Inspeccionar objetos y variables:** puedes utilizar `console.log()` para imprimir el contenido de objetos y variables complejas. Esto te permite examinar su estructura y asegurarte de que contengan los datos esperados. Por ejemplo:



The screenshot shows the Chrome DevTools Console with the 'Console' tab selected. The code being executed is:

```
> const objeto = { nombre: "Ejemplo", edad: 25 };
const variable = 10;

console.log("Datos del objeto:", objeto);
console.log("Valor de la variable:", variable);
```

The output displayed in the console is:

```
Datos del objeto: {nombre: 'Ejemplo', edad: 25}
Valor de la variable: 10
```

The object log shows a detailed view of the object's structure, including its prototype chain.

3. **Establecer puntos de interrupción:** puedes utilizar puntos de interrupción en el código para detener la ejecución en un punto específico y examinar el estado de las variables en ese momento. En las herramientas de desarrollo del navegador, puedes hacer clic en el número de línea a la izquierda del código para establecer un punto de interrupción. Cuando se alcance el punto de interrupción, la ejecución se detendrá y podrás examinar el contexto y las variables en ese punto.
 - Abre las herramientas de desarrollo del navegador. puedes hacerlo haciendo clic derecho en la página y seleccionando "Inspeccionar" o presionando la tecla F12.
 - En la pestaña "Fuentes" o "Sources" de las herramientas de desarrollo, busca el archivo JavaScript en el que deseas establecer el punto de interrupción.

- Haz clic en el número de línea a la izquierda del código para establecer un punto de interrupción. Verás que se agrega un icono de punto rojo en esa línea para indicar que se ha establecido el punto de interrupción.
 - Ejecuta el código normalmente. Cuando se alcance el punto de interrupción durante la ejecución, el navegador se detendrá y la ejecución se pausará en ese punto.
 - En ese momento, puedes examinar el contexto y las variables en la pestaña "Scope" o "Scope Variables" de las herramientas de desarrollo. Allí podrás ver los valores actuales de las variables y realizar cualquier otra acción de depuración necesaria.
 - Utiliza las herramientas de depuración adicionales, como el paso a paso y los botones de control (play, pausa, siguiente) para continuar la ejecución del código después de examinar el estado de las variables.
 - Establecer puntos de interrupción te permite detener la ejecución en puntos clave de tu código para inspeccionar el estado de las variables y diagnosticar problemas. Es una técnica poderosa para depurar y entender cómo se comporta tu programa en diferentes momentos de su ejecución.
4. **Utilizar comandos de depuración:** Las herramientas de desarrollo del navegador también ofrecen comandos y funcionalidades adicionales para depurar el código. Puedes utilizar comandos como `debugger`; para detener la ejecución en un punto específico, `console.trace()` para mostrar la pila de llamadas y `console.error()` para imprimir mensajes de error en la consola.
- **`debugger`:** Puedes insertar la instrucción `debugger` en tu código JavaScript para detener la ejecución en un punto específico. Cuando el navegador encuentra esta instrucción, se pausa la ejecución y se abre automáticamente la pestaña de las herramientas de desarrollo en el punto donde se encuentra la instrucción. Esto te permite inspeccionar el estado de las variables y realizar un seguimiento de la ejecución desde ese punto en adelante.
 - **`console.trace()`:** Puedes utilizar `console.trace()` para imprimir la pila de llamadas en la consola del navegador. Esto te muestra la secuencia de funciones que se han llamado hasta el momento en que se ejecuta este comando. Es útil para rastrear cómo se ha

llegado a un determinado punto en tu código y comprender el flujo de ejecución.

- **console.error():** Puedes utilizar `console.error()` para imprimir mensajes de error en la consola del navegador. Esto te permite señalar problemas o errores específicos en tu código. Los mensajes de error se resaltarán en rojo en la consola, lo que facilita la identificación y solución de problemas.

5. **Utilizar la consola interactiva:** Además de utilizar `console.log()`, puedes escribir código JavaScript directamente en la consola para probar expresiones, invocar funciones y realizar pruebas rápidas. Esto te permite experimentar y verificar el comportamiento del código en tiempo. Para acceder a la consola interactiva, abre las herramientas de desarrollo del navegador y dirígete a la pestaña "Consola" o "Console". Allí encontrarás un campo de entrada donde puedes escribir tu código JavaScript.

- **Probar expresiones:** Puedes escribir expresiones matemáticas o lógicas directamente en la consola para obtener el resultado. Por ejemplo, puedes escribir `2 + 2` y presionar Enter para obtener el resultado 4.
- **Invocar funciones:** Si has definido funciones en tu código, puedes invocarlas directamente desde la consola. Por ejemplo, si tienes una función llamada `saludar()`, puedes escribir `saludar()` y presionar Enter para que se ejecute la función y muestre el resultado en la consola.
- **Acceder a variables y objetos:** Si has definido variables u objetos en tu código, puedes acceder a ellos desde la consola para verificar su contenido o realizar operaciones con ellos. Por ejemplo, si tienes una variable llamada `nombre` con el valor "John", puedes escribir `nombre` y presionar Enter para ver su valor en la consola.
- La consola interactiva es una herramienta muy útil para experimentar y verificar el comportamiento del código en tiempo real. Puedes utilizarla para realizar pruebas rápidas, verificar resultados y explorar características del lenguaje JavaScript. real.

Recuerda que las herramientas de desarrollo pueden variar según el navegador, pero la mayoría de ellos proporcionan funcionalidades similares para depurar el código JavaScript. Aprovecha al máximo la consola y las herramientas de depuración para identificar y solucionar problemas en tu código.

Selectores básicos: getElementById

El **Modelo de Objetos del Documento (DOM)** es una representación en forma de árbol de la estructura de un documento HTML o XML. Proporciona una interfaz para acceder y manipular los elementos del documento, permitiendo a los programadores interactuar con el contenido, la estructura y los estilos de una página web.

Cuando un navegador carga un documento HTML, crea una representación interna del documento en forma de árbol, donde cada elemento HTML se convierte en un nodo en ese árbol. Cada nodo representa un elemento, como etiquetas HTML, atributos, texto, comentarios, etc. Estos nodos se organizan jerárquicamente, reflejando la estructura anidada del documento HTML.

El DOM proporciona una API (Interfaz de Programación de Aplicaciones) que nos permite acceder y manipular estos nodos y sus propiedades. Podemos utilizar métodos y propiedades del DOM para seleccionar elementos, modificar contenido, agregar o eliminar elementos, cambiar estilos, y mucho más. Esto permite crear dinamismo en las páginas web y permite a los desarrolladores crear interactividad y funcionalidades personalizadas.

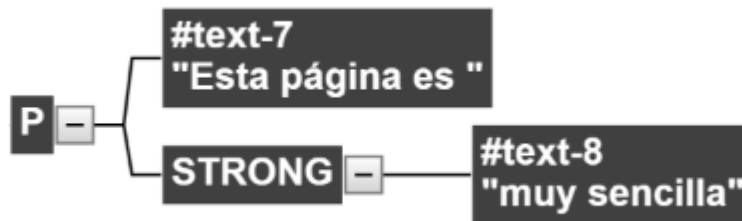
El acceso al DOM se realiza a través del objeto document, que es parte del entorno JavaScript en el navegador. El objeto document representa el documento HTML cargado y proporciona métodos y propiedades para interactuar con él. A partir de ahí, podemos utilizar métodos como getElementById(), querySelector(), createElement(), entre otros, para seleccionar, crear y manipular elementos dentro del DOM.

Estructura del DOM y Nodos

- Cada etiqueta HTML se transforma en un nodo de tipo "Elemento". La conversión se realiza de forma jerárquica.
- De esta forma, del nodo raíz solamente pueden derivar los nodos HEAD y BODY.
- Cada etiqueta HTML se transforma en un nodo que deriva del correspondiente a su "etiqueta padre".
- La transformación de las etiquetas HTML habituales genera dos nodos
 - **Nodo elemento:** correspondiente a la propia etiqueta HTML.
 - **Nodo texto:** contiene el texto encerrado por esa etiqueta HTML.

```
<p>Esta página es <strong>muy sencilla</strong></p>
```

La etiqueta <p> se transforma en los siguientes nodos del DOM:



Acceso por objeto document

```
console.dir(document)
console.dir(document.head)
console.dir(document.body)
```

El acceso a body usando la referencia document.body requiere que el script se incluya al final del <body> en el HTML.

```
<script src="ruta/a1/archivo/script.js"></script>
</body>
```

Tipos de Nodos

La especificación completa de DOM define 12 tipos de nodos, los más usados son:

Document: Nodo raíz del que derivan todos los demás nodos del árbol.

Element: Representa cada una de las etiquetas XHTML. Puede contener atributos y derivar otros nodos de él.

Attr: Se define un nodo de este tipo para representar cada uno de los atributos de las etiquetas HTML, es decir, uno por cada par atributo=valor.

Text: Nodo que contiene el texto encerrado por una etiqueta HTML.

Comment: Representa los comentarios incluidos en la página HTML.

Acceso al DOM

Existen distintos métodos para acceder a los elementos del DOM empleando en la clase Document.

Los más comunes son:

- `getElementsByTagName()`
- `getElementsByClassName()`
- `getElementById()`

Getelementbyid(): El método `getElementById()` sirve para acceder a un elemento de la estructura HTML, utilizando su atributo ID como identificación.

```
//código HTML
<div id="mi-id">
  <p>Lorem</p>
</div>

//código JS
const miElemento = document.getElementById('mi-id');
```

En este ejemplo, se selecciona el elemento con el ID "mi-id" y se almacena en la variable `miElemento`. Al ser un ID único, no devuelve una colección de elementos, sino el elemento específico.

getElementsByTagName(): Este método permite seleccionar elementos por su etiqueta HTML. Recibe como argumento el nombre de la etiqueta y devuelve una colección de elementos que coinciden con esa etiqueta.

```
const elementosP = document.getElementsByTagName('p');
```

En este ejemplo, se seleccionan todos los elementos `<p>` del documento y se almacenan en la variable `elementosP`. La variable contendrá una colección de elementos, y podemos acceder a ellos mediante índices, por ejemplo: `elementosP[0]`, `elementosP[1]`, etc.

getElementsByClassName(): Este método permite seleccionar elementos por su clase CSS. Recibe como argumento el nombre de la clase y devuelve una colección de elementos que tienen esa clase. Por ejemplo:

```
<p class="mi-clase">Este es un párrafo con la clase "mi-clase".</p>  
  
const elementosClase = document.getElementsByClassName('mi-clase');
```

En este ejemplo, se seleccionan todos los elementos que tienen la clase "mi-clase" y se almacenan en la variable `elementosClase`. Al igual que con `getElementsByTagName()`, la variable contendrá una colección de elementos y podemos acceder a ellos mediante índices.

Obtención y manipulación de valores y textos de los elementos del DOM

Innet Text

La propiedad `innerText` de un nodo nos permite modificar su nodo de texto. Es decir, acceder y/o modificar el contenido textual de algún elemento del DOM.

```
const elemento = document.getElementById('mi-elemento');  
  
elemento.innerText = 'Nuevo texto';
```

En este ejemplo, seleccionamos un elemento con el ID "mi-elemento" utilizando `getElementById()`. Luego, asignamos un nuevo valor a la propiedad `innerText` del elemento. Esto reemplaza el contenido textual del elemento con el nuevo valor proporcionado, conservando el formato y las etiquetas HTML.

Es importante destacar que **la propiedad `innerText` solo muestra el texto visible en el elemento** y no incluye el texto oculto por estilos CSS, mientras que la propiedad `textContent` muestra todo el contenido textual, incluyendo el texto oculto. Si necesitas modificar todo el contenido textual, incluyendo el oculto, puedes usar `textContent` en su lugar.

Al utilizar `innerText`, se aplicarán las reglas de escape de HTML, lo que significa que cualquier etiqueta HTML dentro del texto será mostrada como texto plano. Si deseas modificar el contenido HTML completo de un elemento, debes utilizar la propiedad `innerHTML` en su lugar.

Innet HTML

innerHTML permite definir el código html interno del elemento seleccionado. El navegador lo interpreta como código HTML y no como contenido de texto, permitiendo desde un string crear una nueva estructura de etiquetas y contenido.

Al pasar un string con formato de etiquetas html y contenido a través de la propiedad innerHTML, el navegador genera nuevos nodos con su contenido dentro del elemento seleccionado.

```
const elemento = document.getElementById('mi-elemento');  
elemento.innerHTML = '<p>Nuevo contenido HTML</p>';
```

En este ejemplo, seleccionamos un elemento con el ID "mi-elemento" utilizando getElementById(). Luego, asignamos un nuevo valor a la propiedad innerHTML del elemento. El navegador generará nuevos nodos basados en el código HTML proporcionado, en este caso, un párrafo <p> con el texto "Nuevo contenido HTML". El contenido anterior del elemento se reemplazará por esta nueva estructura de etiquetas y contenido.

Es importante tener en cuenta que al utilizar innerHTML, debes asegurarte de que el código HTML proporcionado sea seguro y confiable, ya que cualquier código malicioso podría ser interpretado y ejecutado por el navegador. Si estás obteniendo el código HTML de fuentes externas o no confiables, es recomendable realizar una validación adecuada antes de asignarlo a innerHTML.

También que al utilizar innerHTML, se reevaluará y reconstruirá el contenido HTML del elemento, lo que puede tener un impacto en el rendimiento en comparación con otras formas más simples de modificar el contenido textual.

Class Name

A través de la propiedad className de algún nodo seleccionado podemos acceder al atributo class del mismo y definir cuáles van a ser sus clases:

```
const elemento = document.getElementById('mi-elemento');  
elemento.className = 'clase-1 clase-2';
```

En este ejemplo, seleccionamos un elemento con el ID "mi-elemento" utilizando `getElementById()`. Luego, asignamos un nuevo valor a la propiedad `className` del elemento. En este caso, le estamos asignando dos clases, "clase-1" y "clase-2". Esto reemplazará todas las clases existentes del elemento por estas nuevas clases.

También puedes utilizar `className` para agregar o quitar clases individuales del elemento:

```
const elemento = document.getElementById('mi-elemento');
elemento.className += ' nueva-clase';
```

En este caso, utilizamos el operador `+=` para concatenar la clase "nueva-clase" al atributo `class` existente del elemento, sin eliminar las clases previas.

Es importante destacar que **al utilizar `className`, debes tener en cuenta que estás sobrescribiendo por completo el atributo `class`**. Si necesitas realizar operaciones más complejas con las clases, como agregar o quitar clases específicas, te recomendaría utilizar la propiedad `classList` en su lugar, que ofrece métodos más flexibles para trabajar con las clases de un elemento.

Agregar o quitar Nodos

Creación de elementos

Para crear elementos se utiliza la función `document.createElement()`, y se debe indicar el nombre de etiqueta HTML que representará ese elemento. Luego debe agregarse como hijo el nodo creado con `append()`, al `body` o a otro nodo del documento actual.

```
// Crear un elemento <p>
const nuevoParrafo = document.createElement('p');

// Agregar contenido al elemento
nuevoParrafo.textContent = 'Este es un nuevo párrafo';

// Obtener el nodo padre al que deseas agregar el nuevo elemento
const contenedor = document.getElementById('mi-contenedor');
```



```
// Agregar el nuevo elemento como hijo del nodo padre
contenedor.appendChild(nuevoParrafo);
```

Creamos un nuevo elemento `<p>` utilizando `document.createElement('p')`. Luego, asignamos un contenido de texto al elemento utilizando la propiedad `textContent`. A continuación, seleccionamos un nodo padre existente en el documento con el ID "mi-contenedor" utilizando `getElementById()`. Por último, utilizamos el método `appendChild()` para agregar el nuevo elemento como hijo del nodo padre.

Eliminar elementos

Para quitar nodos del DOM, puedes utilizar el método `removeChild()` para eliminar un nodo específico o utilizar `parentNode.removeChild()` para eliminar un nodo hijo de su nodo padre.

```
// Obtener el nodo que deseas eliminar
const nodoEliminar = document.getElementById('mi-nodo');

// Obtener el nodo padre del nodo a eliminar
const nodoPadre = nodoEliminar.parentNode;

// Eliminar el nodo del DOM
nodoPadre.removeChild(nodoEliminar);
```

En este caso, seleccionamos el nodo que deseamos eliminar con el ID "mi-nodo" utilizando `getElementById()`. Luego, obtenemos el nodo padre del nodo a eliminar utilizando la propiedad `parentNode`. Finalmente, utilizamos el método `removeChild()` en el nodo padre para eliminar el nodo del DOM.

El método `remove()` te permite eliminar un nodo seleccionado del DOM. Puedes llamar a este método en el nodo que deseas eliminar y se eliminará del árbol DOM.

```
const elemento = document.getElementById('mi-elemento');  
elemento.remove();
```

En este ejemplo, seleccionamos un elemento con el ID "mi-elemento" utilizando `getElementById()`. Luego, llamamos al método `remove()` en el elemento seleccionado. Esto eliminará el elemento y todos sus descendientes del DOM.

Es importante destacar que al utilizar `remove()`, el nodo eliminado no se puede recuperar, por lo que debes tener cuidado al utilizar este método y asegurarte de que realmente desees eliminar el nodo del DOM.

Hay que tener en cuenta que **el método `remove()` no es compatible con todos los navegadores**, especialmente versiones antiguas. Si necesitas compatibilidad con navegadores más antiguos, puedes utilizar otras técnicas para eliminar nodos del DOM, como utilizar el método `parentNode.removeChild()`.

Obtener datos de Inputs

Para obtener o modificar datos de un formulario HTML desde JS, podemos hacerlo mediante el DOM. Accediendo a la propiedad `value` de cada input seleccionado:

```
<form id="mi-formulario">  
  <label for="nombre">Nombre:</label>  
  <input type="text" id="nombre" />  
  
  <label for="email">Email:</label>  
  <input type="email" id="email" />  
  
  <button type="button" onclick="mostrarDatos()">Mostrar Datos</button>  
</form>
```

En este ejemplo, tenemos un formulario con dos campos de entrada: uno para el nombre y otro para el email. Luego, tenemos un botón que al hacer clic invoca la función `mostrarDatos()`.

```
function mostrarDatos() {  
    // Obtener los valores de los campos de entrada  
    const nombre = document.getElementById('nombre').value;  
    const email = document.getElementById('email').value;  
  
    // Mostrar los datos en la consola o realizar alguna acción con ellos  
    console.log('Nombre:', nombre);  
    console.log('Email:', email);  
}
```

En la función `mostrarDatos()`, utilizamos `getElementById()` para obtener los elementos de entrada del formulario por su ID. Luego, accedemos a la propiedad `value` de cada elemento para obtener su valor actual. Puedes utilizar estos valores para mostrarlos en la consola, realizar alguna acción adicional con ellos, enviarlos a un servidor, etc.

Al obtener los datos de los campos de entrada, puedes validarlos antes de utilizarlos para asegurarte de que cumplan con los requisitos necesarios (por ejemplo, validar el formato del email, verificar que no estén vacíos, etc.).

Query Selector

El método `querySelector()` nos permite seleccionar nodos del DOM utilizando selectores CSS. Funciona de manera similar a cómo seleccionamos elementos con CSS en nuestros estilos.

```
<div id="mi-elemento" class="clase1 clase2">  
    <p>Contenido del elemento</p>  
</div>  
  
// Seleccionar el elemento por su ID
```

```
const elementoID = document.querySelector('#mi-elemento');  
  
// Seleccionar el elemento por su clase  
const elementoClase = document.querySelector('.clase1');  
  
// Seleccionar el elemento por su etiqueta  
const elementoEtiqueta = document.querySelector('p');
```

En este ejemplo, utilizamos `querySelector()` para seleccionar diferentes elementos del DOM. En el primer caso, seleccionamos el elemento con el ID "mi-elemento" utilizando el selector `#mi-elemento`. En el segundo caso, seleccionamos el elemento con la clase "clase1" utilizando el selector `.clase1`. En el tercer caso, seleccionamos el primer elemento `<p>` que encontramos en el documento.

`querySelector()` devuelve el primer elemento que coincide con el selector especificado. Si hay varios elementos que coinciden con el selector, solo se seleccionará el primero.

Puedes utilizar selectores más complejos en `querySelector()` para seleccionar elementos de manera más precisa. Por ejemplo, puedes utilizar selectores combinados como `.clase1.clase2` para seleccionar elementos que tengan ambas clases, o selectores de descendencia como `#mi-elemento p` para seleccionar elementos `<p>` que sean descendientes del elemento con el ID "mi-elemento".

Query Selector All

Query Selector me retorna el primer elemento que coincida con el parámetro de búsqueda, o sea un sólo elemento. Si quiero obtener una colección de elementos puede utilizar el método `querySelectorAll()` siguiendo el mismo comportamiento.

```
<ul>  
  <li>Elemento 1</li>  
  <li>Elemento 2</li>  
  <li>Elemento 3</li>  
</ul>
```

```
// Seleccionar todos los elementos <li> dentro del <ul>
const elementos = document.querySelectorAll('ul li');

// Iterar sobre la colección de elementos
elementos.forEach((elemento) => {
  console.log(elemento.textContent);
});
```

En este ejemplo, **utilizamos `querySelectorAll()` para seleccionar todos los elementos `<li>` que se encuentran dentro de un `<ul>`**. Esto devuelve una colección de nodos que coinciden con el selector especificado.

Puedes utilizar `querySelectorAll()` con cualquier selector CSS válido, como selectores de etiqueta, clases, ID, atributos, etc. La colección de nodos devuelta se puede iterar utilizando métodos como `forEach()` para realizar acciones en cada elemento.

Eventos básicos: `onClick` y `onChange` 🧑

Los eventos son acciones o sucesos que ocurren en el navegador, como hacer clic en un elemento, ingresar texto en un campo de entrada o cambiar el valor de una casilla de verificación. JavaScript nos permite manejar estos eventos y ejecutar código en respuesta a ellos.

JavaScript permite asignar una función a cada uno de los eventos. Reciben el nombre de event handlers o manejadores de eventos. Así, ante cada evento, JavaScript asigna y ejecuta la función asociada al mismo. Hay que entender que los eventos suceden constantemente en el navegador.

JavaScript lo que nos permite hacer es escuchar eventos sobre elementos seleccionados. Cuando escuchamos el evento que esperamos, se ejecuta la función que definimos en respuesta.

A esta escucha se la denomina event listener.

Definir eventos en JS

Opción 1

El método `addEventListener()` permite definir qué evento escuchar sobre cualquier elemento seleccionado.

El primer parámetro corresponde al nombre del evento y el segundo a la función de respuesta.

El método **`addEventListener()`** es utilizado para asignar un evento a un elemento específico en JavaScript. Permite definir qué evento escuchar y qué función se debe ejecutar como respuesta a ese evento.

```
<button id="mi-boton">Haz clic aquí</button>

const boton = document.getElementById('mi-boton');

boton.addEventListener('click', function() {
  console.log('¡Hola, mundo!');
});
```

En este ejemplo, hemos seleccionado un elemento de botón con el ID "mi-boton" utilizando `getElementById()`. Luego, utilizamos `addEventListener()` para asignar el evento click al botón y proporcionamos una función anónima como respuesta al evento. Dentro de la función, se imprimirá el mensaje "¡Hola, mundo!" en la consola cuando el botón sea clicado.

La ventaja de utilizar `addEventListener()` es que puedes asignar múltiples eventos a un mismo elemento y manejarlos de forma separada. Por ejemplo, puedes asignar tanto el evento click como el evento mouseover a un elemento y ejecutar diferentes funciones en respuesta a cada evento.

también puedes utilizar la sintaxis de funciones flecha para definir la función de respuesta de manera más concisa:

```
boton.addEventListener('click', () => {
  console.log('¡Hola, mundo!');
});
```

De esta manera, `addEventListener()` se convierte en una herramienta poderosa para controlar la interacción del usuario con tu aplicación y definir acciones personalizadas para diferentes eventos.

Es importante tener en cuenta que al utilizar `addEventListener()`, la función de respuesta se ejecutará cada vez que ocurra el evento especificado.

Opción 2

Otra forma de definir eventos en JavaScript es utilizando las propiedades de los nodos para asignar las respuestas a los eventos. Estas propiedades se identifican con el nombre del evento precedido por el prefijo "on".

```
<button id="mi-boton">Haz clic aquí</button>

const boton = document.getElementById('mi-boton');

boton.onclick = function() {
  console.log('¡Hola, mundo!');
};
```

En este ejemplo, hemos seleccionado **un elemento de botón con el ID "mi-boton" utilizando `getElementById()`**. Luego, hemos asignado la función anónima como respuesta al evento `onclick` utilizando la propiedad **`onclick` del botón**. Cuando se haga clic en el botón, se imprimirá el mensaje "¡Hola, mundo!" en la consola.

También puedes utilizar funciones flecha para definir la respuesta de manera más concisa:

```
boton.onclick = () => {
  console.log('¡Hola, mundo!');
};
```

Esta forma de asignar eventos utilizando las propiedades del nodo puede ser útil en ciertos casos. Sin embargo, ten en cuenta que al utilizar esta sintaxis, si asignas un nuevo manejador de eventos a la misma propiedad, el manejador anterior se reemplazará. Por lo tanto, esta forma no es tan flexible como `addEventListener()` cuando se trata de asignar múltiples eventos o manejar eventos de forma más dinámica.

Es importante mencionar que aunque ambas opciones son válidas, se recomienda utilizar `addEventListener()` en lugar de asignar directamente a las propiedades de eventos. `addEventListener()` proporciona más flexibilidad y permite manejar múltiples eventos de forma separada sin sobrescribir los manejadores existentes.

Evento Clic y Doble Clic

El evento clic (`click`) se activa cuando se hace clic en un elemento, ya sea mediante un clic del mouse o una pulsación táctil en dispositivos móviles. El evento doble clic (`dblclick`) se activa cuando se realiza un doble clic en un elemento, es decir, dos clics rápidos consecutivos.

```
<button id="myButton">Haz clic o doble clic aquí</button>
```

```
const button = document.getElementById('myButton');
```

```
button.addEventListener('click', () => {  
  console.log('Has hecho clic en el botón');  
});
```

```
button.addEventListener('dblclick', () => {  
  console.log('Has hecho doble clic en el botón');  
});
```

En este ejemplo, cuando se hace clic en el botón, se activa el **evento click** y se muestra un mensaje en la consola. Cuando se realiza un doble clic en el botón, se activa el **evento dblclick** y se muestra otro mensaje en la consola.

Eventos del mouse

Se producen por la interacción del usuario con el mouse. Entre ellos se destacarán los que se encuentran a continuación.

- **mousemove:** El movimiento del mouse sobre el elemento activa el evento.
- **click:** Se activa después de mousedown o mouseup sobre un elemento válido.
- **mousedown/mouseup:** Se oprime/suelta el botón del ratón sobre un elemento.
- **mouseover/mouseout:** El puntero del mouse se mueve sobre/sale del elemento.

```
//CODIGO HTML DE REFERENCIA  
<button id="btnMain">CLICK</button>  
  
//CODIGO JS  
let boton = document.getElementById("btnMain")  
boton.onclick = () => {console.log("Click")}  
boton.onmousemove = () => {console.log("Move")}
```

Eventos del teclado

Se producen por la interacción del usuario con el teclado. Entre ellos se destacarán los que se encuentran a continuación.

- **keydown:** Cuando se presiona.
- **keyup:** Cuando se suelta una tecla.

```
//CODIGO HTML DE REFERENCIA  
<input id = "nombre" type="text">  
<input id = "edad" type="number">  
  
//CODIGO JS  
let input1 = document.getElementById("nombre")
```

```
let input2 = document.getElementById("edad")

input1.onkeyup = () => {console.log("keyUp")}

input2.onkeydown = () => {console.log("keyDown")}
```

Evento change

El **evento change** se activa cuando se detecta un cambio en el valor de un elemento, pero no se dispara inmediatamente mientras se está escribiendo en un campo de texto. En cambio, se activa cuando el elemento pierde el foco, es decir, cuando el usuario realiza alguna acción para mover el foco a otro elemento, como hacer clic fuera del campo de texto o presionar la tecla "Tab" para cambiar al siguiente campo.

Este evento es útil cuando se necesita detectar cambios en el valor de un elemento, como un input de texto o un select, después de que el usuario ha finalizado su interacción con el elemento. Al capturar el evento change, puedes realizar acciones adicionales o procesar los nuevos valores ingresados por el usuario.

```
<input type="text" id="inputText">

const input = document.getElementById('inputText');

input.addEventListener('change', (event) => {
  const newValue = event.target.value;

  console.log('Nuevo valor:', newValue);

  // Realizar acciones adicionales con el nuevo valor
});
```

En este ejemplo, cuando el usuario realiza un cambio en el campo de texto y luego lo abandona (por ejemplo, haciendo clic fuera del campo), se activa el evento change y se captura el nuevo valor ingresado. puedes realizar acciones

adicionales con ese nuevo valor, como enviarlo a una base de datos, realizar cálculos o actualizar otros elementos en la página.

```
<select id="selectOptions">
  <option value="option1">Opción 1</option>
  <option value="option2">Opción 2</option>
  <option value="option3">Opción 3</option>
</select>

const select = document.getElementById('selectOptions');

select.addEventListener('change', (event) => {
  const selectedValue = event.target.value;
  console.log('Valor seleccionado:', selectedValue);
  // Realizar acciones adicionales con el valor seleccionado
});
```

En este caso, cuando el usuario selecciona una opción diferente en el elemento select, se activa el evento change y se obtiene el valor seleccionado.

El evento change solo se activa cuando hay un cambio real en la selección y se confirma, es decir, cuando se elige una opción diferente. Si el usuario selecciona la misma opción nuevamente, no se activará el evento change.

Evento Input

El evento input se activa cada vez que se realiza un cambio en el valor de un elemento de entrada, como un campo de texto o un área de texto. A diferencia del evento change, el evento input se dispara inmediatamente mientras el usuario está interactuando con el elemento, incluso mientras está escribiendo o eliminando texto.

El evento **input** es útil cuando se necesita detectar cambios en tiempo real a medida que el usuario escribe o realiza acciones de edición en un campo de texto. puedes utilizar este evento para realizar validaciones en tiempo real, actualizar vistas previas, realizar búsquedas instantáneas o cualquier otra acción que requiera una respuesta inmediata a medida que el usuario modifica el valor.

```
<input type="text" id="inputText">

const input = document.getElementById('inputText');

input.addEventListener('input', (event) => {
  const currentValue = event.target.value;
  console.log('Valor actual:', currentValue);
  // Realizar acciones adicionales con el valor actual
});
```

En este ejemplo, cada vez que el usuario escribe o elimina texto en el campo de texto, se activa el evento **input** y se captura el valor actual del campo. puedes realizar acciones adicionales con ese valor, como mostrar una vista previa de los cambios, realizar cálculos en tiempo real o actualizar otros elementos en la página.

El evento **input** se dispara en cada cambio, por lo que puede haber una gran cantidad de eventos generados mientras el usuario está interactuando con el campo de texto. Si necesitas realizar acciones que no requieren una respuesta inmediata, como realizar una búsqueda o una solicitud a un servidor, es posible que desees combinar el evento **input** con un temporizador o utilizar otro enfoque para evitar realizar múltiples solicitudes innecesarias.

Evento Submit

El evento submit se activa cuando se envía un formulario, ya sea haciendo clic en un botón de envío o presionando la tecla Enter dentro de un campo de entrada. Este evento se utiliza comúnmente para capturar y procesar los datos ingresados por el usuario antes de enviarlos al servidor.

```
<form id="myForm">

  <label for="name">Nombre:</label>

  <input type="text" id="name" required>

  <button type="submit">Enviar</button>

</form>

const form = document.getElementById('myForm');

form.addEventListener('submit', (event) => {
  event.preventDefault();

  // Evitar el envío del formulario por defecto

  // Obtener los valores de los campos de entrada
  const name = document.getElementById('name').value;

  // Realizar acciones adicionales con los datos ingresados
  console.log('Nombre:', name);

  // Hacer algo más con los datos (enviar a un servidor, procesarlos, etc.)
});
```

En este ejemplo, cuando el usuario hace clic en el botón de envío dentro del formulario, **se activa el evento submit y se ejecuta la función de respuesta**. Dentro de esta función, puedes capturar los valores de los campos de entrada y realizar acciones adicionales, como validar los datos, procesarlos o enviarlos a un servidor.

La llamada a **event.preventDefault()** evita que el formulario se envíe de forma predeterminada, lo que permite realizar acciones personalizadas antes de enviar los datos. Puedes usar esta función para realizar validaciones adicionales, mostrar mensajes de error o realizar cualquier otra acción necesaria antes de enviar los datos.

```
<form id="loginForm">

  <label for="email">Email:</label>

  <input type="email" id="email" required>

  <label for="password">Contraseña:</label>

  <input type="password" id="password" required>

  <button type="submit">Iniciar sesión</button>

</form>

const form = document.getElementById('loginForm');

form.addEventListener('submit', (event) => {

  event.preventDefault(); // Evitar el envío del formulario por defecto

  // Obtener los valores de los campos de entrada

  const email = document.getElementById('email').value;

  const password = document.getElementById('password').value;

  // Validar las credenciales ingresadas

  if (email === 'ejemplo@example.com' && password === '12345') {

    alert('Inicio de sesión exitoso');
```

```
// Realizar acciones adicionales después de iniciar sesión

} else {

    alert('Credenciales incorrectas. Por favor, intenta nuevamente.');
```



```
// Realizar acciones adicionales si las credenciales son incorrectas

}

});
```

En este ejemplo, cuando el usuario hace clic en el botón "Iniciar sesión" dentro del formulario de inicio de sesión, se activa el evento submit y se ejecuta la función de respuesta. Dentro de esta función, se obtienen los valores de los campos de email y contraseña.

Luego se realiza la validación de las credenciales ingresadas. Si el email y la contraseña coinciden con los valores esperados (en este caso, "ejemplo@example.com" y "12345" respectivamente), se muestra un mensaje de inicio de sesión exitoso. Si las credenciales no coinciden, se muestra un mensaje de credenciales incorrectas.

Cierre 🖊️

En esta unidad hemos explorado los fundamentos del lenguaje de programación JavaScript, desde sus bases y breve historia hasta su relevancia en el desarrollo web. Hemos aprendido sobre la importancia de **las variables** en JavaScript y cómo utilizarlas para almacenar y manipular datos en nuestros programas. Además, hemos explorado las **expresiones aritméticas y las sentencias condicionales**, que nos permiten realizar cálculos y tomar decisiones en nuestros scripts.

También hemos profundizado en **el concepto de funciones**, que son bloques de código reutilizables que nos permiten encapsular tareas y ejecutarlas en diferentes momentos. Hemos **aprendido cómo ejecutar código JavaScript en la consola del navegador y cómo utilizar la consola para depurar y encontrar errores en nuestro código**.

En cuanto al manejo del DOM, hemos **explorado los selectores básicos, como getElementById, que nos permiten acceder y manipular los elementos de una página web**. Hemos visto cómo obtener y modificar valores y textos de los elementos del DOM, así como crear, eliminar y modificar nodos.

Por último, hemos abordado los eventos básicos, como onClick, onSubmit y onChange, y cómo asignar funciones como respuesta a estos eventos para interactuar con los usuarios de nuestras aplicaciones.

Referencias

Wikipedia. (s.f.). No te repitas. https://es.wikipedia.org/wiki/No_te_repitas

Wikipedia. (s.f.). Principio KISS. https://es.wikipedia.org/wiki/Principio_KISS

Wikipedia. (s.f.). YAGNI. <https://es.wikipedia.org/wiki/YAGNI>

JavaScript.info. (s.f.). Árbol del Modelo de Objetos del Documento (DOM).
<https://es.javascript.info/dom-nodes>

JavaScript.info. (s.f.). Recorriendo el DOM.
<https://es.javascript.info/dom-navigation>

JavaScript.info. (s.f.). Propiedades de los nodos.
<https://es.javascript.info/basic-dom-node-properties>

¡Muchas gracias!

Nos vemos en la próxima lección

